Anleitung: CORA

RPA-Potenzialanalyse von BPMN-Aktivitäten mit Large Language Models

Inhalt

Was ist CORA? **1**

Was du brauchst **2**

Schritt 1: Docker installieren **2**
- Windows . . . 2
- macOS . . . 2
- Linux . . . 3
- Prüfen, ob Docker funktioniert . . . 3

Schritt 2: CORA herunterladen **3**

Schritt 3: API-Key eintragen **3**

Schritt 4: Datensatz vorbereiten **4**

Schritt 5: CORA starten **4**

Schritt 6: CORA bedienen **5**
- 6.1 Das LLM auswählen (Reiter “LLM Settings”) . . . 5
- 6.2 Ein einzelnes Prozessmodell analysieren . . . 5
- 6.3 Mit dem LLM über das Ergebnis sprechen (Reiter “Chat”) . . . 5
- 6.4 Einen Datensatz evaluieren (Reiter “Evaluation”) . . . 6

Schritt 7: CORA beenden und wieder starten **6**

Wenn etwas nicht funktioniert **6**

Was ist CORA?

CORA (Criteria-Oriented RPA Assessment) ist ein Programm, das BPMN-Prozessmodelle mit Hilfe eines Large Language Models (LLM) untersucht. Für jede Aktivität eines Prozesses schätzt das LLM ein, ob sie sich mit Robotic Process Automation (RPA) automatisieren lässt und welche der neun RPA-Eignungskriterien nach Bédard et al. sie erfüllt.

CORA kann zwei Dinge:

- **Ein einzelnes Prozessmodell analysieren:** Du lädst eine `.bpmn`-Datei hoch und siehst im Diagramm, welche Aktivitäten das LLM als automatisierbar einstuft.
- **Einen ganzen Datensatz evaluieren:** CORA analysiert alle Prozessmodelle eines Datensatzes und vergleicht die Ergebnisse mit euren eigenen Labels. Daraus berechnet es Kennzahlen wie Precision, Recall und F1-Score.

CORA läuft mit **Docker**. Du musst also keine Programmiersprache und keine Entwicklungswerkzeuge installieren, nur Docker.

Was du brauchst

- Einen Computer mit Windows 10/11 (64 Bit), macOS oder Linux. Empfohlen sind mindestens 8 GB Arbeitsspeicher und 10 GB freier Speicherplatz.
- Eine Internetverbindung.
- Einen **OpenRouter-API-Key**. Den bekommt ihr von eurer Betreuung. Über diesen Schlüssel greift CORA auf die LLMs zu, und über ihn werden die Kosten abgerechnet. Gib ihn nicht weiter.
- Den **CORA-Ordner** (als ZIP-Datei oder von GitHub, siehe Schritt 2).
- Eure **Prozessmodelle** (`.bpmn`) und, für die Evaluation, die passenden **Label-Dateien** (`.labels.json`). Die Labels erstellt ihr mit dem CORA-Labeling-Tool, dafür gibt es eine eigene Anleitung.

Schritt 1: Docker installieren

Docker ist ein Programm, das andere Programme in abgeschlossenen "Containern" ausführt. Alles, was CORA braucht, steckt in diesen Containern. Für die Arbeit an der Universität (Ausbildung) und für private Zwecke ist **Docker Desktop kostenlos**.

Windows

1. Öffne https://www.docker.com/products/docker-desktop/ und klicke auf **Download for Windows**. Wenn du die Wahl hast, nimm **AMD64** (das passt für fast alle Rechner). **ARM64** brauchst du nur bei Geräten mit Snapdragon-Prozessor.
2. Starte die heruntergeladene Datei `Docker Desktop Installer.exe` mit einem Doppelklick.
3. Lass im Installationsfenster die Option **"Use WSL 2 instead of Hyper-V"** angehakt und klicke dich mit **OK** durch.
4. Starte den Computer neu, wenn das Installationsprogramm dich dazu auffordert.
5. Öffne **Docker Desktop** über das Startmenü. Beim ersten Start musst du die Nutzungsbedingungen akzeptieren. Eine Anmeldung bei Docker ist nicht nötig, du kannst diesen Schritt überspringen.
6. Falls Docker meldet, dass **WSL** aktualisiert werden muss: Öffne das Startmenü, tippe `PowerShell`, klicke mit der rechten Maustaste auf **Windows PowerShell** und wähle **Als Administrator ausführen**. Gib dort `wsl --update` ein und drücke Enter. Starte danach Docker Desktop neu.
7. Warte, bis Docker Desktop unten links **"Engine running"** anzeigt (grün). Erst dann ist Docker bereit.

Hinweis: Wenn Docker beim Start meldet, dass die **Virtualisierung** deaktiviert ist, muss sie im BIOS/UEFI des Computers eingeschaltet werden (Stichwort "Intel VT-x" bzw. "AMD-V"). Wie das geht, hängt vom Hersteller ab. Suche nach "Virtualisierung aktivieren" und dem Namen deines Laptops, oder frag deine Betreuung.

macOS

1. Finde heraus, welchen Prozessor dein Mac hat: Klicke oben links auf das Apple-Symbol und dann auf **Über diesen Mac**. Steht dort **Apple M1, M2, M3 …**, brauchst du die Version für **Apple Silicon**. Steht dort **Intel**, brauchst du die Version für **Intel**.
2. Öffne https://www.docker.com/products/docker-desktop/ und lade die passende Version herunter.
3. Öffne die heruntergeladene `.dmg`-Datei und ziehe **Docker** in den Ordner **Programme**.

4. Starte Docker aus dem Ordner **Programme**, bestätige die Sicherheitsabfragen und akzeptiere die Nutzungsbedingungen.
5. Warte, bis Docker Desktop **“Engine running”** anzeigt.

Linux

Installiere entweder Docker Desktop für Linux oder die Docker Engine mit dem Compose-Plugin. Die Anleitungen für die einzelnen Distributionen findest du unter https://docs.docker.com/engine/install/.

Prüfen, ob Docker funktioniert

Öffne ein **Terminal**:

- **Windows:** Startmenü öffnen, `PowerShell` tippen, **Windows PowerShell** öffnen.
- **macOS:** `Cmd + Leertaste` drücken, `Terminal` tippen, Enter.

Gib nacheinander diese beiden Befehle ein und drücke jeweils Enter:

```
docker --version
docker compose version
```

Wenn bei beiden eine Versionsnummer erscheint, ist Docker bereit. Wenn eine Fehlermeldung kommt, ist Docker Desktop wahrscheinlich nicht gestartet.

Schritt 2: CORA herunterladen

- **Als ZIP-Datei von der Betreuung:** Entpacke die Datei (Rechtsklick → **Alle extrahieren** bzw. Doppelklick auf dem Mac).
- **Von GitHub:** Öffne die Projektseite, klicke auf den grünen Button **Code** und dann auf **Download ZIP**. Entpacke die Datei.

Lege den entpackten Ordner an einen einfachen Ort, zum Beispiel `Dokumente\CORA`. **Vermeide Ordner, die mit OneDrive, iCloud oder Dropbox synchronisiert werden**, weil die Synchronisation Dateien während der Auswertung blockieren kann.

Im Ordner findest du unter anderem:

Datei / Ordner	Bedeutung
`compose.yaml`	Startanleitung für Docker (nicht verändern)
`.env.example`	Vorlage für deine Einstellungen
`dataset/`	Hier kommen eure Prozessmodelle und Labels hinein
`backend/`, `frontend/`	Programmcode von CORA (nicht verändern)

Schritt 3: API-Key eintragen

CORA liest deinen API-Key aus einer Datei namens `.env` im CORA-Ordner.

1. **Dateiendungen sichtbar machen (nur Windows):** Öffne den Explorer, klicke oben auf **Ansicht** → **Einblenden** und setze den Haken bei **Dateinamenerweiterungen**. Sonst siehst du nicht, ob eine Datei wirklich `.env` heißt oder heimlich `.env.txt`. *Auf dem Mac* sind Dateien, die mit einem Punkt beginnen, versteckt. Mit `Cmd + Shift + .` (Punkt) blendest du sie im Finder ein.
2. Kopiere die Datei `.env.example` und benenne die Kopie in **`.env`** um. Bestätige die Warnung zur Dateiendung mit **Ja**.

3. Öffne `.env` mit einem einfachen Texteditor (Windows: Rechtsklick → **Öffnen mit** → **Editor**; Mac: **TextEdit**).
4. Ersetze `<your OpenRouter API key>` durch deinen Key. Die Anführungszeichen bleiben stehen:

   ```
   OPENROUTER_API_KEY="sk-or-v1-..."
   ```
5. Trage bei `DATASET_NAME` einen Namen für euren Datensatz ein, zum Beispiel `"Gruppe3"`. Unter diesem Namen erscheint der Datensatz später in CORA und in den Ergebnisdateien.
6. Speichere die Datei. Achte darauf, dass sie weiterhin `.env` heißt (nicht `.env.txt`).

Wichtig: Die `.env`-Datei enthält deinen persönlichen Schlüssel. Schicke sie niemandem und lade sie nirgendwo hoch.

Schritt 4: Datensatz vorbereiten

CORA wertet immer **genau einen Datensatz** aus: den Inhalt des Ordners `dataset/`.

Für jedes Prozessmodell brauchst du zwei Dateien mit **demselben Namen**:

- `<Name>.bpmn`: das Prozessmodell (z. B. aus Signavio als BPMN 2.0 XML exportiert)
- `<Name>.labels.json`: eure Labels für dieses Modell

Das Labeling-Tool speichert Labels unter dem Namen `<Name>.labels.<Annotator-ID>.json`, zum Beispiel `Urlaubsantrag.labels.anna.json`. CORA liest aber nur Dateien, die genau `<Name>.labels.json` heißen. Kopiere deshalb für jedes Modell die Label-Datei, die als Referenz gelten soll (z. B. eure gemeinsam abgestimmte Fassung), und benenne die Kopie in `<Name>.labels.json` um. Die Dateien der einzelnen Annotatoren dürfen daneben liegen bleiben, CORA ignoriert sie.

Beispiel für einen fertigen `dataset`-Ordner:

```
dataset/
  Urlaubsantrag.bpmn
  Urlaubsantrag.labels.json
  Urlaubsantrag.labels.anna.json
  Urlaubsantrag.labels.ben.json
  Rechnungsprüfung.bpmn
  Rechnungsprüfung.labels.json
```

Die Dateien müssen **direkt** im Ordner `dataset/` liegen, nicht in einem Unterordner.

Hinweise:

- Fehlt zu einem Modell die `.labels.json`, behandelt CORA alle Aktivitäten dieses Modells als “nicht automatisierbar”. Die Kennzahlen für dieses Modell sind dann falsch.
- Für die Analyse einzelner Modelle (Schritt 6.2) brauchst du keine Labels.
- Wenn du den Inhalt von `dataset/` oder die `.env` änderst, führe danach im Terminal noch einmal `docker compose up -d` aus (siehe Schritt 5), damit CORA die Änderung übernimmt.

Schritt 5: CORA starten

1. Stelle sicher, dass **Docker Desktop läuft** (“Engine running”).
2. Öffne ein Terminal **im CORA-Ordner**:
   - **Windows:** Öffne den CORA-Ordner im Explorer. Klicke oben in die Adressleiste (dort, wo der Pfad steht), tippe `powershell` und drücke Enter.
   - **macOS:** Öffne das Terminal, tippe `cd` (mit Leerzeichen danach), ziehe den CORA-Ordner aus dem Finder in das Terminalfenster und drücke Enter.
3. Gib diesen Befehl ein und drücke Enter:

```
docker compose up --build -d
```

Beim **ersten Start** lädt Docker alles Nötige herunter und baut CORA zusammen. Das dauert einige Minuten. Fertig ist es, wenn die Ausgabe stoppt und Zeilen mit **Started** oder **Running** erscheinen.

4. Öffne im Browser die Adresse **http://localhost:8081**.

CORA läuft jetzt auf deinem eigenen Rechner. Die Adresse funktioniert nur dort und nur, solange Docker Desktop läuft.

Schritt 6: CORA bedienen

Die Oberfläche besteht aus dem **Diagrammbereich** (links bzw. oben) und einer **Seitenleiste** mit drei Reitern: **LLM Settings**, **Chat** und **Evaluation**.

6.1 Das LLM auswählen (Reiter “LLM Settings”)

1. Wähle unter **LLM Model Name** das Modell aus, mit dem du arbeiten möchtest.
2. Die Felder **LLM Base URL** und **API Key** kannst du so lassen. Ist das Feld *API Key* leer, verwendet CORA den Key aus deiner `.env`-Datei.
3. **Seed**, **Temperature** und **Top P** steuern, wie zufällig die Antworten des LLMs sind. Lass die Voreinstellungen stehen, außer eure Aufgabe verlangt etwas anderes.
4. Klicke auf **Apply Settings**.

Wichtig: Ohne Klick auf **Apply Settings** arbeitet CORA weiter mit dem vorher eingestellten Modell. Nach dem Neuladen der Seite stehen wieder die Voreinstellungen drin. Kontrolliere die Auswahl deshalb vor jeder Analyse.

6.2 Ein einzelnes Prozessmodell analysieren

1. Klicke oben auf **Upload** und wähle eine `.bpmn`-Datei aus. Das Diagramm erscheint.
2. Klicke auf **Analyze BPMN** und warte, bis die Analyse fertig ist (meist einige Sekunden bis eine Minute).
3. Aktivitäten, die das LLM als **automatisierbar** einstuft, werden **grün** eingefärbt.
4. Fahre mit der Maus über eine grüne Aktivität. Ein Fenster zeigt die erfüllten Kriterien und die Begründung des LLMs.
5. Das **Info-Symbol** (ganz rechts in der Werkzeugleiste) zeigt die neun Kriterien mit Beispielen.

Weitere Buttons:

Button	Funktion
Download Report	Speichert das Ergebnis der Analyse als Datei `analysis_report.json` (in deinem Download-Ordner)
Download	Speichert das aktuelle Diagramm als `.bpmn`-Datei
Clear BPMN	Entfernt die Einfärbung der Analyse
New	Öffnet ein leeres Diagramm, in dem du selbst modellieren kannst

6.3 Mit dem LLM über das Ergebnis sprechen (Reiter “Chat”)

Nach einer Analyse kannst du im Reiter **Chat** Fragen zum Ergebnis stellen, zum Beispiel: “Warum ist die Aktivität *Antrag prüfen* nicht automatisierbar?” Das LLM kennt dabei das zuletzt analysierte Modell. Mit dem Button **Clear** leerst du den Verlauf.

6.4 Einen Datensatz evaluieren (Reiter “Evaluation”)

1. Prüfe im Reiter **LLM Settings**, dass das richtige Modell eingestellt ist (siehe 6.1).
2. Wechsle zum Reiter **Evaluation**.
3. Trage bei **Number of Repetitions** ein, wie oft jedes Prozessmodell ausgewertet werden soll (1 bis 5). Mehrere Wiederholungen zeigen, wie stabil die Ergebnisse des LLMs sind.
4. Unter **Select Datasets** steht der Name aus `DATASET_NAME`.
5. Klicke auf **Apply Evaluation**.

Eine Evaluation kann je nach Größe des Datensatzes, Modell und Anzahl der Wiederholungen **von wenigen Minuten bis zu mehreren Stunden** dauern. Beachte dabei:

- **Der Fortschrittsbalken ist nur eine Schätzung.** Er bleibt bei 97 % stehen, bis die Evaluation wirklich fertig ist. Das ist normal.
- **Lass den Computer nicht in den Ruhezustand gehen** und schließe den Browser-Tab nicht. Unter Windows stellst du das unter **Einstellungen → System → Energie** ein, auf dem Mac unter **Systemeinstellungen → Batterie** bzw. **Energie sparen**.
- **Den echten Fortschritt** siehst du im Terminal mit:

  ```
  docker compose logs -f backend
  ```

  Dort erscheint nach jedem Prozessmodell eine Zeile wie `--- progress: repetition 2, BPMN model 7/20`. Mit `Strg + C` (Mac: `Ctrl + C`) beendest du nur die Anzeige, die Evaluation läuft weiter.
- **Jede Evaluation kostet Guthaben** auf dem API-Key. Teste deshalb zuerst mit **1 Wiederholung** und wenigen Modellen, bevor du einen großen Lauf startest.

Ergebnisse: Nach dem Ende zeigt CORA die Kennzahlen an (u. a. Precision, Recall, F1-Score, Accuracy und den Anteil vollständig korrekt bewerteter Modelle). Frühere Evaluationen findest du unter **Evaluation History**.

Zusätzlich speichert CORA jede Evaluation als Datei im CORA-Ordner:

- `backend/src/evaluation/evaluation-runs/`: das **vollständige Protokoll** mit allen Antworten des LLMs, den Einstellungen und möglichen Fehlern. Diese Dateien braucht ihr für eigene Auswertungen und Fehleranalysen.
- `backend/src/evaluation/evaluation-history/`: die Ergebnisse, die CORA in der *Evaluation History* anzeigt.

Schritt 7: CORA beenden und wieder starten

Beenden: Im Terminal im CORA-Ordner:

```
docker compose down
```

Danach kannst du auch Docker Desktop schließen. Eure Ergebnisdateien bleiben erhalten.

Wieder starten: Docker Desktop öffnen, warten bis “Engine running” erscheint, dann im CORA-Ordner:

```
docker compose up -d
```

Wenn du eine neue Version von CORA bekommen hast, verwende wieder `docker compose up --build -d`.

Wenn etwas nicht funktioniert

Problem	Lösung
`docker` wird nicht erkannt	Docker Desktop ist nicht installiert oder nicht gestartet. Starte Docker Desktop und öffne danach ein **neues** Terminal.
“Cannot connect to the Docker daemon” oder “error during connect”	Docker Desktop starten und warten, bis “Engine running” angezeigt wird.
http://localhost:8081 lädt nicht	Ein bis zwei Minuten warten und neu laden. Mit `docker compose ps` prüfen, ob beide Container laufen (Status “running”).
“port is already allocated” beim Start	Ein anderes Programm nutzt Port 8081. Ändere in `compose.yaml` die Zeile `"8081:80"` zu `"8091:80"` und öffne dann http://localhost:8091.
Unter *Select Datasets* steht nichts	Prüfe, ob die Dateien direkt in `dataset/` liegen (nicht in einem Unterordner), und führe `docker compose up -d` aus. Lade danach die Seite neu.
Fehler mit “401” oder “No auth credentials”	Der API-Key in `.env` ist falsch oder fehlt. Korrigieren, speichern, dann `docker compose up -d`.
Im Log steht “429 … rate-limited”	Der Anbieter des Modells ist kurz überlastet. CORA wartet automatisch und versucht es erneut. Steht danach `Test case "…" failed`, wurde dieses Prozessmodell nicht bewertet. Notiert es und sprecht mit eurer Betreuung, wie ihr es nachholt.
Im Log steht “404 … deprecated”	Das Modell wird nicht mehr angeboten. Wähle ein anderes Modell und sprich das mit eurer Betreuung ab.
Nach einer langen Evaluation zeigt CORA einen Fehler an	Prüfe den Ordner `backend/src/evaluation/evaluation-runs/`. Ist dort eine neue Datei, ist die Evaluation trotzdem vollständig gespeichert.
Docker meldet, dass Virtualisierung fehlt	Siehe Hinweis in Schritt 1 (Windows).

Bei weiteren Fragen meldet euch per Mail bei maurice.cordts@uni-ulm.de. Schickt dabei, wenn möglich, die Fehlermeldung aus dem Terminal oder einen Screenshot mit.